

ОКО — сопроводительная документация

Автоматизированный сервис поиска нарушений на строительных площадках Москвы ·
Департамент градостроительной политики · конкурс «Лидеры цифровой трансформации
2026» · версия 2.1, 29.09.2026

Документ отвечает на требования раздела 5 ТЗ: **модель данных** (раздел 3), **код** (раздел 5), **условия и ограничения** (раздел 6), **инструкции по сборке, установке и запуску** (раздел 7). Разделы 1, 2 и 4 описывают решение и логику сопоставления снимков с графиком. Исходный код, методика и демо-данные — в репозитории; полное описание архитектуры — [backend/readme.md](#).

1. Назначение и состав решения

ОКО берёт снимки с камер стройплощадки, находит на них строительную технику, относит её к зоне и этапу календарного графика по методике «этап работ → необходимая техника» и выдаёт отклонения с объяснением: какая техника ожидалась, какая видна, в какой зоне и на каких снимках. Инженер подтверждает отклонение — оно становится нарушением в реестре с номером, подрядчиком и сроком устранения.

Обязательная функция ТЗ	Как реализована
Обнаружение и классификация техники	Детектор RF-DETR Large из ML-части: 10 классов, mAP50 0,907. Версия v2 добавляет каток и грузовик из перечня ТЗ по открытым датасетам из ссылок организаторов (до её обучения — 6 из 8 типов ТЗ). Для камер общего плана — детекция по фрагментам 3×3. Детектор сменный: PyTorch, ONNX на CPU, удалённый AI-сервис на GPU, демо-разметка.
Сопоставление с графиком	Методика на все 377 видов работ справочника организаторов: строка графика → вид работ (по коду или названию) → профиль техники → проверки на снимках зоны за день. 46 профилей задают обязательную, парную и допустимую технику и наблюдаемость работ с камер.
Визуализация результатов	Веб-интерфейс: сводка по зонам, снимки с рамками и объяснением, отклонения со снимками-доказательствами, график, методика, проверка своего снимка. REST API со Swagger и консольное приложение.
Выявление отклонений	10 правил: нет данных, работы не ведутся, нет необходимой техники, неполный комплект (пример ТЗ), техники меньше плана, опережение, этап не завершён в срок, техника не соответствует этапу, техника в зоне без работ, стадия по снимку не совпадает с планом.
Проверка и прогноз ИИ (дополнительно)	Выводы детектора, модели готовности, локальной VLM и правил сводятся в один контекст и уходят в YandexGPT (Yandex AI Studio): вердикт по каждой рамке и каждому отклонению, пропущенная техника, стадия, прогноз и рекомендации. Исправления инженер применяет кнопкой и может отменить.

Состав репозитория

Папка	Содержимое
backend/	Сервис: FastAPI, движок правил, детекторы, ИИ-анализ, модель данных, консоль, 71 автотест, Dockerfile
frontend/app/	Интерфейс прототипа — одна страница без сборки, отдаётся бэкендом

Папка	Содержимое
methodology/	Методика: классы техники, профили, привязка видов работ, правила; XLSX для экспертов; справочник организаторов
training/	ML-часть: модель готовности и стадии (шаги 0–4), дообучение детектора v2 и экспорт для CPU (шаги 5–8), 6 автотестов
data/demo/	Два демо-проекта: графики, камеры и зоны, 31 снимок с эталонной разметкой
docs/	Документация, презентация, рекомендации по установке камер, иллюстрации
tools/	Сборка методики из справочника и генерация демо-данных
weights/	Веса моделей: RF-DETR (вне git), голова модели готовности — <code>weights/readiness/прогон/</code> (13 МБ)
docker-compose.yml	PostgreSQL + сервис; профиль <code>gpu</code> — AI-сервис с RF-DETR на видеокарте
deploy/yandex-cloud/	Публичное демо на BM Yandex Cloud: RF-DETR, модель готовности и VLM на CPU, YandexGPT

2. Архитектура



Рис. 1. Компоненты прототипа и их взаимодействие

Прототип — один сервис на CPU (модульный монолит). Отдельно выносятся только детектор, когда ему нужен GPU: тот же код, запущенный с `ОКО_ДЕТЕКТОР=rfdetr`, обслуживает эндпоинт `/internal/detect`, а основной сервис обращается к нему с `ОКО_ДЕТЕКТОР=http`.

Модуль	Задача
<code>app/api/*</code>	REST: проекты, график, зоны и камеры, снимки, отклонения, нарушения, методика, быстрая проверка, контракт AI-сервиса
<code>app/services.py</code>	Сценарии: импорт графика, приём снимка, пересчёт отклонений зоны за день, вердикт инженера, сводка
<code>app/engine.py</code>	Движок правил — чистые функции без БД и ввода-вывода: окно зоны за день и этапы → отклонения и объяснение
<code>app/methodology.py</code>	Загрузка методики, синонимы классов, сопоставление строки графика со справочником
<code>app/schedule_io.py</code>	Разбор XLSX/CSV графика по заголовкам столбцов, иерархия этапов, техника из графика

Модуль	Задача
<code>app/detection/*</code>	Детекторы с общим интерфейсом: RF-DETR (PyTorch), ONNX Runtime, AI-сервис по HTTP, демо-разметка; детекция по фрагментам
<code>app/imaging.py</code>	Время съёмки, контроль качества, превью и разметка снимков-доказательств
<code>app/zones.py</code>	Точка опоры рамки → зона на кадре
<code>app/models.py</code> , <code>app/db.py</code>	Модель данных (SQLAlchemy 2), PostgreSQL или SQLite
<code>app/cli.py</code>	Консольное приложение: проверка снимка, правила вида работ, загрузка папки снимков, отчёт
<code>app/ai/*</code>	ИИ-анализ: модель готовности (только инференс), локальная VLM, клиент YandexGPT, сбор контекста, сведение слоёв и исправления, фоновые задачи

Конвейер обработки снимка

1. **Приём.** Файл через интерфейс, API или консоль. SHA-256: повторный снимок не обрабатывается дважды.
2. **Время съёмки.** Поле загрузки → EXIF → имя файла (`CAM-01_2026-09-24_10-30.jpg`) → время загрузки. OCR даты на кадре (ML-часть) — запасной источник в промышленной схеме.
3. **Контроль качества** до детекции: яркость 0,12–0,93, контраст $\geq 0,035$, резкость (дисперсия лапласиана) ≥ 25 . Непригодный снимок даёт «нет данных», а не «нет техники».
4. **Детекция** RF-DETR один раз на снимок. Если детектор снимок не распознал (демо-детектор без весов и чужой снимок) — это тоже «нет данных». Для камер общего плана — кадр целиком плюс сетка 3×3, с фрагментов берутся только мелкие рамки (< 2 % кадра), всё сливается через NMS.
5. **Зона.** Машина относится к зоне по точке опоры — середине нижней стороны рамки; башенный кран обслуживает всю площадку.
6. **Объяснение по снимку:** этапы зоны на дату, ожидаемая и видимая техника, пары, лишняя техника.
7. **Пересчёт дня зоны** по всем её снимкам: отклонения обновляются по ключу, снимки-доказательства прикладываются.
8. **Стадия и готовность** для кадров общего плана — модель готовности в фоне; **ИИ-анализ** снимка — по кнопке или автоматически (раздел 4).

Приём снимка без детекции (качество, зоны, объяснение, пересчёт дня зоны) занимает около 16 мс на CPU — медиана на 31 демо-снимке. Предупреждение появляется сразу после приёма снимка; подтверждение — со вторым снимком серии.

3. Модель данных

Данные хранятся в реляционной БД: PostgreSQL в Docker и в эксплуатации, SQLite на ноутбуке. Схема одна (SQLAlchemy 2), 20 таблиц в пяти группах. Сложные структуры (требования профиля, полигоны зон, расчёт отклонения) — в JSON-полях (JSONB в PostgreSQL).

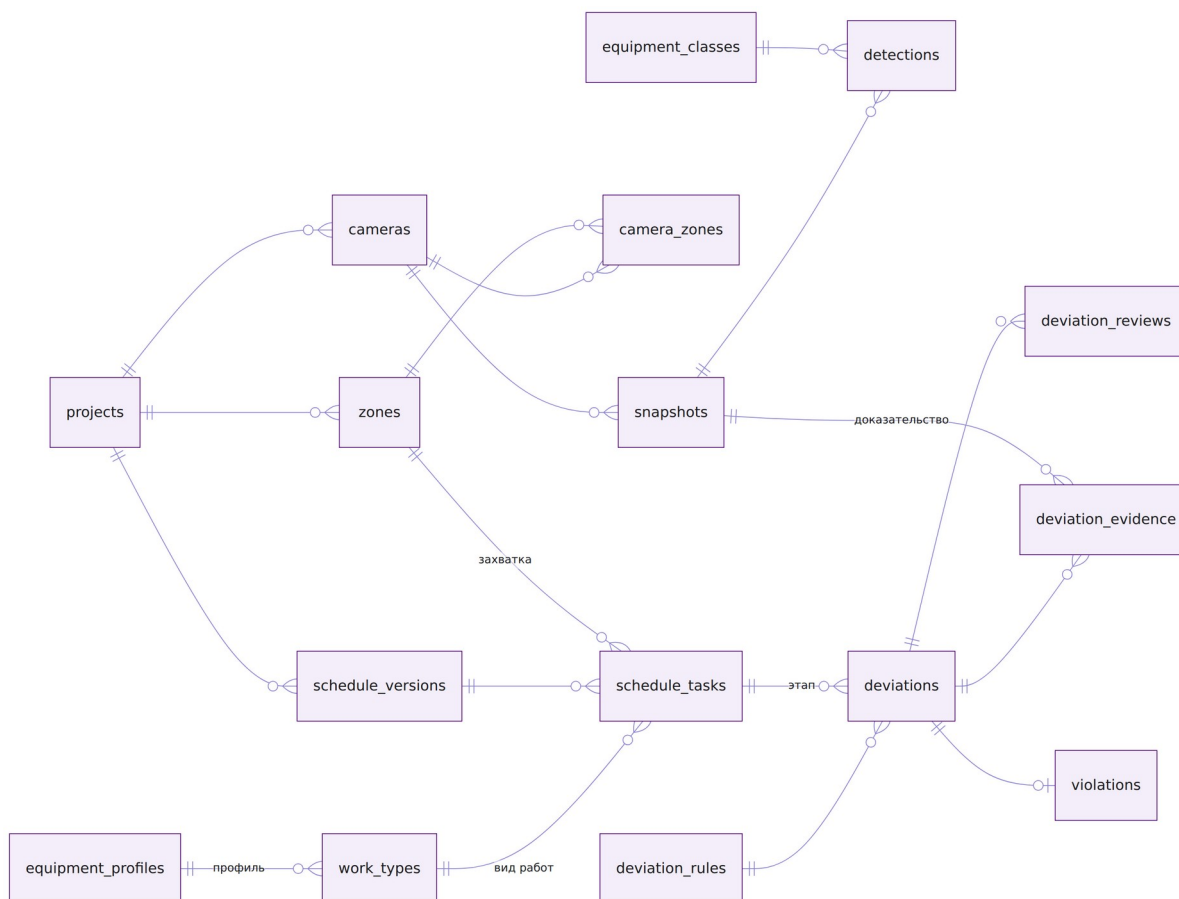


Рис. 2. Связи таблиц; поля — в таблицах ниже, полная ER-диаграмма — docs/assets/oko_data_model.png

Техника и методика

Таблица	Ключевые поля	Что хранит
equipment_classes	key, name, in_tz, site_wide, models, color	Классы техники: 8 из ТЗ + 4 своих + 2 в плане; в каких версиях детектора класс есть
object_types	key, name	9 типов объектов справочника: жильё, образование, дороги...
equipment_profiles	id, observability, required, companions, allowed, not_detected, is_composite	Профиль техники: обязательная (роль, классы, минимум, окно «снимок» или «смена»), парная, допустимая; наблюдаемость
work_types	id, code, name, level, parent_id, stage, profile_id, observability, object_types, expert_note	377 видов работ справочника с профилем, стадией S1–S5 и замечанием экспертов
deviation_rules	code, title, kind, severity, confirmed_severity, message, recommendation, rules_version	10 правил: важность по одному снимку и по серии, шаблоны предупреждения и рекомендации

Этапы работ: проект, площадка, график

Таблица	Ключевые поля	Что хранит
projects	name, object_type, address, timezone	Объект строительства
zones	key, name, color	Зоны площадки = захватки графика

Таблица	Ключевые поля	Что хранит
<code>cameras, camera_zones</code>	<code>source_url, is_overview, tiles, default_zone_id; polygon</code>	Камеры и полигоны зон на кадре каждой камеры (в долях кадра)
<code>site_equipment</code>	<code>equipment_class, zone_keys, active_from, active_to</code>	Заявленная техника площадки (башенный кран вне кадра зон)
<code>schedule_versions</code>	<code>version, source_filename, is_baseline, rows_total, rows_matched</code>	Версии графика: базовая сохраняется, новая загрузка — новая версия
<code>schedule_tasks</code>	<code>wbs, name, work_type_id, match_method, match_score, profile_id, zone_id, start_date, end_date, parent_task_id, is_summary, planned_equipment, contractor</code>	Этап графика с привязкой к виду работ (по коду, названию или вручную), зоной, сроками, техникой из графика и подрядчиком

Наблюдения: снимки и техника

Таблица	Ключевые поля	Что хранит
<code>snapshots</code>	<code>camera_id, taken_at, time_source, sha256, brightness, contrast, sharpness, quality_ok, detector, assessment, processing_ms</code>	Снимок, источник времени, контроль качества, версия модели, стадия и готовность по кадру общего плана
<code>detections</code>	<code>snapshot_id, equipment_class, confidence, x1, y1, x2, y2, zone_id, source, is_rejected</code>	Рамки техники с зоной; <code>is_rejected</code> — рамка, которую инженер признал ложной
<code>snapshot_checks</code>	<code>snapshot_id, zone_key, payload</code>	Объяснение сопоставления снимка с этапами зоны

Выводы: отклонения и нарушения

Таблица	Ключевые поля	Что хранит
<code>deviations</code>	<code>key (UNIQUE с project_id), day, zone_id, task_id, rule_code, equipment_class, severity, status, message, recommendation, metrics, first_seen, last_seen, snapshots_count, rules_version, review_status</code>	Отклонение: одно на день, зону, этап, правило и класс; «предварительно» или «подтверждено»; расчёт; версия правил
<code>deviation_evidence</code>	<code>deviation_id, snapshot_id, detection_ids, note</code>	Снимки-доказательства и подсвечиваемые рамки
<code>deviation_reviews</code>	<code>verdict, comment, reviewer, created_at</code>	Журнал вердиктов инженера, только добавление
<code>violations</code>	<code>number, deviation_id, zone_id, task_id, contractor, severity, status, due_date, closed_at</code>	Реестр нарушений: HP-2026-0001 , подрядчик из графика, срок устранения
<code>ai_reviews</code>	<code>kind, status, step, vlm_model, vlm_output, llm_provider, llm_model, llm_output, context, final, applied, tokens_in, tokens_out, duration_ms</code>	ИИ-анализ снимка или дня: что видела и ответила каждая модель, что ушло в LLM, сведение слоёв, применённые и отменённые исправления

Жизненный цикл отклонения

1. Снимок принят → пересчёт дня зоны → отклонение создаётся или обновляется по ключу `день | зона | этап | правило | класс` (upsert), поэтому повторная обработка не плодит дубли.
2. Один снимок — «предварительно»; серия снимков дня или очень уверенная рамка ($\geq 0,8$) для правил лишней техники — «подтверждено». Важность по серии может быть выше: «внимание» → «критично».
3. Инженер выносит вердикт: **подтвердить** — создаётся нарушение; **ложное срабатывание** — рамки помечаются `is_rejected` и не участвуют в пересчёте (в промышленной схеме — выгрузка для дообучения); **форс-мажор** — фиксируется в журнале.
4. Пересчёт не отменяет решений: рассмотренное отклонение не удаляется и не возвращается в работу.

Методика хранится и в файлах (`methodology/*.yaml` — источник правды, редактируется экспертами), и в БД (синхронизируется при старте). Версия правил пишется в каждое отклонение, поэтому любой вывод воспроизводим.

4. Логика сопоставления снимков с этапами графика



Рис. 3. Снимок → зона и дата → этапы графика → профиль техники → предупреждение (пример из ТЗ)

Связь снимка с графиком строится через два ключа — **зону** (полигон на кадре камеры = захватка графика) и **дату снимка**. Для зоны на дату движок берёт этапы трёх видов: идущие (требования их профилей), предстоящие в ближайшие 21 день (опережение) и завершившиеся в последние 30 дней (затягивание). Всё, что не нужно идущим этапам зоны, — лишняя техника с указанием этапа, к которому она относится.

Профиль техники

Каждый вид работ справочника ссылается на профиль. Профиль отвечает на четыре вопроса:

- **Какая техника обязательна** — группы «достаточно любого класса из списка» с минимумом и окном: `snapshot` — на каждом снимке, `shift` — хотя бы раз за смену (подвоз, бетонирование захватками, доставка).
- **Какая техника работает в паре** — «ведущая → парная»: экскаватор → самосвал, автобетоносмеситель → насос или кран, грейдер → каток. Это пример из ТЗ, возведённый в правило.
- **Какая техника допустима** — вспомогательная техника, появление которой нормально.
- **Насколько работы видны камере** — наблюдаемость определяет, что проверяется.

EXCAVATION:	# вид работ 12.3.1 «Устройство котлована»
-------------	---


```

observability: high
required:
  - {role: разработка грунта, any_of: [excavator], min: 1, window: snapshot}
  - {role: вывоз грунта, any_of: [dump_truck], min: 1, window: shift}
companions:
  - lead: [excavator]
    partner: [dump_truck]
  message: Экскаватор работает без самосвалов — грунт не вывозится, ...
  allowed: [bulldozer, loader]

```

Наблюдаемость	Видов работ	Что проверяется
высокая	70	Работы ведёт тяжёлая техника на открытой площадке: нет техники — нет работ; проверка по каждому снимку
средняя	99	Техника эпизодическая (подача, монтаж, доставка): нужна хотя бы раз за смену
низкая	70	Техника вспомогательная: проверяется только лишняя техника
не видно с камер	138	Работы внутри здания и документы: только лишняя тяжёлая техника

Правила выявления отклонений

Код	Отклонение	1 снимок	Серия	Когда срабатывает
NO_DATA	Нет данных	к сведению	к сведению	Все снимки зоны непригодны: темно, туман, засветка, размыто
NO_EQUIPMENT	Работы не ведутся	внимание	критично	Этап с видимой техникой идёт, а зона пуста (серия — не меньше часа)
MISSING_REQUIRED	Нет необходимой техники	внимание	критично	Нет техники для обязательной роли профиля
INCOMPLETE_SET	Неполный комплект техники	внимание	критично*	Ведущая техника без парной на ≥ 30 % снимков — пример ТЗ
INSUFFICIENT_COUNT	Техники меньше плана	внимание	критично	В графике «2 × миксер», а одновременно в кадре не больше одного
EARLY_START	Опережение графика	к сведению	к сведению	Техника этапа зоны, который начнётся в пределах 21 дня
STAGE_OVERRUN	Этап не завершён в срок	внимание	внимание	Техника этапа зоны, закончившегося не раньше 30 дней назад
UNEXPECTED_EQUIPMENT	Техника не соответствует этапу	внимание	внимание	Техника не нужна ни одному идущему этапу зоны
NO_ACTIVE_STAGE	Техника в зоне без работ	внимание	внимание	В зоне по графику ничего не идёт
STAGE_MISMATCH	Стадия не совпадает с планом	внимание	внимание	Модель готовности видит на кадре общего плана стадию раньше плановой

* «Критично», если парной техники не было за весь день; иначе «внимание».

Как снижается число ложных предупреждений

- **Асимметричные пороги уверенности:** для правил лишней техники нужна уверенная рамка ($\geq 0,5$), для правил «техники нет» засчитывается и сомнительная ($\geq 0,3$). Сомнительная рамка не создаёт тревоги ни в одну сторону.
- **Контроль качества до детекции:** ночь, туман, засветка — «нет данных», зона не оценивается.
- **Окна подтверждения:** самосвал мог уехать за грунтом на 10 минут — отклонение подтверждается серией снимков.
- **Техника, которую модель не видит, не требуется:** требование бетононасоса видно в карточке этапа, но в проверку не входит, пока класса нет в модели.
- **Максимум по камерам:** две камеры, видящие один экскаватор в зоне, дают один экскаватор.
- **Заявленная техника:** башенный кран вне кадра зон закрывает требования «нужен кран».
- **Сводные этапы:** если у строки графика есть подэтапы, проверяются подэтапы, а не группа.

ИИ-анализ: все модели и слои → YandexGPT

Дополнительный слой поверх правил: модели проверяют друг друга, а YandexGPT сопоставляет их выводы с графиком и методикой. Анализ снимка — фоновая задача (кнопка «Анализ ИИ» в карточке снимка или `ОКО_AI_AUTO=1`), каждый шаг и ответ каждой модели сохраняются в [ai_reviews](#).

Слой	Что даёт	Где работает
RF-DETR и правила	Рамки техники с зонами; этапы графика, проверки профилей, отклонения	сервис, CPU
Модель готовности	Стадия S1–S5, готовность 0–100 %, план по графику (доля срока), статус «опережаем / успеваем / риск / отстаём» и отставание в днях, карта внимания 9×16	сервис, CPU ≈ 30 с
Локальная VLM	Описание сцены, техника с рамками, стадия, люди, условия съёмки	Qwen3-VL 8B → 4B → 2B → SmolVLM2-500M: самая крупная, что помещается в память
Сверка детектора и VLM	По IoU рамок: совпали, спорный класс, только VLM, только детектор	сервис
YandexGPT	Вердикт по каждой рамке и отклонению, пропущенная техника с рамкой, стадия, новые находки, прогноз, рекомендации — JSON по схеме	Yandex AI Studio

YandexGPT не принимает изображения, поэтому «глаза» конвейера — детектор, модель готовности и VLM. Модель [qwen3.6-35b-a3b](#) из AI Studio получает ещё и сами кадры; Claude и ChatGPT подключаются той же настройкой. Итог сводится по слоям: техника по классам (детектор / VLM / LLM) с расхождениями, стадия по графику, модели, VLM и LLM. «Применить исправления» исключает ложные рамки, исправляет класс и добавляет пропущенную технику, если уверенность LLM не ниже 0,6; правила пересчитываются по снимку и дню, «Отменить» возвращает всё как было. Анализ дня даёт статус площадки, риски срыва сроков с вероятностью, прогноз задержек по этапам и места, где модели и правила, вероятно, ошиблись.

5. Код

- **Движок — чистые функции.** `engine.evaluate_window()` получает наблюдения зоны за день и этапы, возвращает отклонения и объяснение. Нет БД и ввода-вывода — каждый вывод воспроизводим и покрыт тестами на сценариях.
- **Методика — данные, а не код.** Новый вид работ, профиль или порог меняется в YAML без правки Python; `tools/build_methodology.py` проверяет, что у каждого вида работ есть профиль, а в профилях только известные классы.
- **Детектор за интерфейсом.** `Detector.detect(image, tiles)` → `DetectResult` и множество распознаваемых классов. Правила «техники нет» проверяют только классы текущей модели, поэтому смена модели не ломает логику.
- **Настройки — переменные окружения** (`app/config.py`, `.env.example`), значения по умолчанию подходят для ноутбука.
- **Комментарии** — в модулях и там, где логика нетривиальна: пороги, окна подтверждения, слияние камер, сопоставление по названию.
- **Тесты** — 77: движок правил на сценариях (включая пример ТЗ), все демо-сценарии end-to-end, API, детекция по фрагментам и ONNX-декодер, ИИ-конвейер с подменой YandexGPT и VLM (признаки модели готовности сверяются с кодом обучения), сборка датасета и метрики AP50.

backend/app/	
main.py	FastAPI: роутеры, статика интерфейса, /api/health
config.py	настройки из окружения
models.py, db.py	модель данных, сессии
services.py	сценарии: график, приём снимка, пересчёт, вердикт, сводка
engine.py	движок правил (чистые функции)
methodology.py	методика, синонимы классов, сопоставление со справочником
schedule_io.py	разбор XLSX/CSV графика
imaging.py	время, качество, превью, разметка
zones.py	точка опоры → зона
detection/	base, rfdetr_backend, onnx_backend, http_backend, demo, tiling
ai/	readiness, vlm, llm, context, fusion, jobs, prefetch
api/	reference, projects, snapshots, deviations, analyze, ai
cli.py, demo.py	консоль, загрузка демо-проектов
backend/tests/	test_engine, test_demo_scenarios, test_api, test_detection,
test_review_ai	

6. Условия и ограничения

На каких данных обучены модели

Модель	Данные и метрики
Детектор v1	RF-DETR Large, веса ML-части команды: 10 классов, mAP50 0,907, mAP50-95 0,767, precision 0,938 (README ML-части).
Детектор v2	Дообучение от v1 (training/05-06) на открытых датасетах из ссылок организаторов: №1 Construction Equipment (Kaggle, 17 классов, CC0) — основной источник катка и грузовика; №2 arh-df (Kaggle, GPL 2); №3 Construction Machines Images (GitHub, 223 экскаватора, только исследования); №4 Construction Vehicle Images (Kaggle, CC BY-NC-ND 4.0); №5 construction-machinery (Roboflow, 78 снимков, CC BY 4.0). Классы датасетов приводятся к классам ОКО по синонимам, дубликаты между источниками убираются по SHA-256 и перцептивному хешу.

Модель	Данные и метрики
Своя тестовая выборка	Отдельный тестовый набор организаторы не дают. Скрипт 5 размечает каждый снимок по условиям съёмки — день, сумерки, ночь, зима (снег), дальний и ближний план — и делит данные со стратификацией по классу и освещённости; скрипт 7 считает AP50, точность и полноту по каждому классу и условию. Метрики v2 появятся после обучения; в документах приведены только метрики v1.
Готовность и стадия	Замороженные DINOv2 ViT-g и RF-DETR + обучаемая голова на 4 692 кадрах четырёх таймлапсов строек: средняя ошибка готовности 0,9 п.п., точность стадии S1–S5 99,7 %. В сервис подключён прогон на пяти таймлапах (5 892 кадра): на кадрах обучения ошибка 1,2 п.п., стадия 99,7 %; отложенного видео в этом прогоне нет, на новых объектах ошибка будет выше. Модель обучена на общих планах: на кадре зоны её оценка ориентировочная.

Какие типы техники распознаются

Класс	Перечень ТЗ	v1	v2	Примечание
Экскаватор	да	да	да	
Самосвал	да	да	да	
Каток	да	—	да	датасет №1, №5
Кран-манипулятор	да	да	да	
Автобетоносмеситель	да	да	да	
Бульдозер	да	да	да	
Грузовик	да	—	да	датасет №1: грузовик, прицеп, «Газель», цистерна
Автокран	да	да	да	
Погрузчик, грейдер, буровая установка	—	да	да	расширение перечня ТЗ (раздел 6.2 допускает)
Башенный кран	—	да	да	обслуживает всю площадку, к зоне не привязывается
Автобетононасос, асфальтоукладчик	—	—	—	план v3; требования видны, но не проверяются

Для каких этапов настроена логика сопоставления

Для всех 377 видов работ справочника организаторов по 9 типам объектов (жильё, образование, здравоохранение, спорт, культура, административные здания, ДОУ, офисно-деловые центры, дороги). Каждому виду работ назначены профиль, стадия S1–S5 и наблюдаемость; замечания экспертов из справочника учтены явно.

Наблюдаемость	S1	S2	S3	S4	S5	Всего
высокая — каждый снимок	30	15	4	15	6	70
средняя — за смену	12	15	26	12	34	99
низкая — лишняя техника	11	5	14	29	10	70*
не видно с камер	4	0	1	62	71	138

* включая корневую группу «12 Выполнение СМР» без стадии. По технике проверяются 169 видов работ с высокой и средней наблюдаемостью: подготовка территории, снос, котлован и его ограждение, сваи, засыпка, основания, бетонирование, монолит, монтаж краном, наружные сети, дороги, благоустройство. Для фасада и отделки, где техники мало, работает правило стадии по кадру общего плана.

Ограничения прототипа

- Демо-снимки синтетические (ТЗ, раздел 6.2, это допускает); рамки на них — эталонная разметка, её читает демо-детектор. Реальные снимки распознаются, когда подключены веса RF-DETR.
- Детектор v2 обучается скриптами репозитория; до обучения сервис работает с v1 (без катка и грузовика — 6 из 8 типов ТЗ), и правила с этими классами автоматически не проверяются.
- Стадию и готовность считает модель готовности в самом сервисе, если установлены PyTorch и transformers (образ [Dockerfile.ai](#)): на CPU ≈ 30 с на кадр, поэтому оценка идёт в фоне. В демо-проектах стадия берётся из разметки — сцены синтетические.
- ИИ-анализ — подсказка инженеру, а не решение: исправления YandexGPT применяются кнопкой и отменяются, всё отправленное в LLM сохраняется. Для всех слоёв на CPU нужно 16 ГБ памяти; слой, которому памяти не хватает, пропускается с объяснением. VLM на CPU отвечает за 1–2 минуты.
- Без весов модели демо-детектор знает только демо-снимки: свой снимок получает «нет данных» с подсказкой подключить веса, а не ложное «работы не ведутся».
- Зоны на кадре задаются вручную полигонами при настройке камеры; сдвиг камеры меняет зоны — нужен эталонный кадр (см. рекомендации по камерам).
- Счёт одновременной техники — по одному кадру; отслеживание одной машины между снимками (трекинг) и машино-часы — следующий слой.
- Вне рамок: охрана труда (каска, жилеты), объёмы работ в натуральных единицах, расчёты с подрядчиками.

7. Сборка, установка и запуск

Требования

	Сервис и правила	+ RF-DETR на CPU	+ ИИ-анализ на CPU	Весь ML-пайплайн
GPU	не нужен	не нужен	не нужен	NVIDIA 24 ГБ (без Qwen3-VL — от 8 ГБ)
Оперативная память	2 ГБ	8 ГБ	16 ГБ (Qwen3-VL-2B), 32 ГБ (4B)	32 ГБ
ПО	Docker или Python 3.10+	+ ONNX Runtime или PyTorch CPU	+ transformers; доступ к Yandex AI Studio	драйвер NVIDIA ≥ 570, CUDA 12.8
ОС	Windows, Linux, macOS	Windows, Linux, macOS	Linux (Docker)	Windows или Linux

Вариант 1. Docker — одна команда

```
git clone https://github.com/Antonooof/Intelligent-construction-site-monitoring-system.git
cd Intelligent-construction-site-monitoring-system
docker compose up --build
# интерфейс: http://localhost:8000          Swagger: http://localhost:8000/docs
```

Поднимаются PostgreSQL и сервис; демо-проекты загружаются при первом старте. Детектор в режиме **auto**: если в **weights/** лежит **rfdetr_v2.onnx**, распознаются реальные снимки, иначе работает эталонная разметка демо-проектов.

Вариант 2. Ноутбук без Docker

```
scripts/run_local.sh          # Linux, macOS
scripts/run_local.bat         # Windows
# вручную:
python -m venv .venv && source .venv/bin/activate
pip install -r backend/requirements.txt
cd backend && uvicorn app.main:app --port 8000
```

По умолчанию используется SQLite в **data/runtime/oko.db**. Для PostgreSQL задайте **OKO_DATABASE_URL=postgresql+psycopg://user:pass@host:5432/oko**.

Подключение детектора

Режим	Как включить	Где работает
RF-DETR, PyTorch	веса в weights/rfdetr_large_best_ema.pth ; pip install torch torchvision и -r backend/requirements-ml.txt ; OKO_DETECTOR=rfdetr	CPU или GPU
RF-DETR, ONNX	python training/08_export_cpu.py (веса v2, если обучены, иначе v1) → weights/rfdetr_v2.onnx ; OKO_DETECTOR=onnx	CPU без PyTorch; в Docker подхватывается сам
AI-сервис на GPU	OKO_DETECTOR=http docker compose --profile gpu up --build	основной сервис на CPU, детекция на GPU
Демо	OKO_DETECTOR=demo	без весов, только демо-снимки

Публичное демо в Yandex Cloud с ИИ-анализом

```
deploy/yandex-cloud/create-vm.sh  # VM 4 vCPU / 16 ГБ, сервисный аккаунт с ролью
ai.languageModels.user
deploy/yandex-cloud/deploy.sh <IP> # код и веса на VM, сборка, запуск → http://<IP>
deploy/yandex-cloud/upgrade-vm.sh # уже созданная VM: 16 ГБ, сервисный аккаунт,
закреплённый IP
```

К VM привязывается сервисный аккаунт с ролью **ai.languageModels.user**: сервис берёт IAM-токен из сервиса метаданных, и ключ API для YandexGPT не нужен; каталог подставляется из **ус config**. Веса DINOv2 и VLM скачиваются с Hugging Face в фоне после запуска. Подробно — **deploy/yandex-cloud/README.md**.

Демонстрация (6 минут)

- Методика** — вкладка «Методика»: 377 видов работ → 46 профилей; вид работ 12.3.1 и его правило; выгрузка в XLSX.
- График** — 17 строк, 16 сопоставлены со справочником; «Монтаж башенного крана» — привязать вручную кнопкой «Указать вид работ».
- Сводка 24.09.2026** — 8 отклонений, 2 критичных; доска зон с последним снимком и проверками.
- Пример ТЗ** — «Котлован, секция 1»: экскаватор без самосвалов, 2 снимка-доказательства, «возможно снижение темпа работ».

5. **Снимок с объяснением** — ожидалось «экскаватор + самосвал», видно «экскаватор», пара нарушена.
6. **Вердикт** — «Подтвердить → нарушение» создаёт НР-2026-0001; в зимнем проекте бульдозер (уборка снега) — «Ложное срабатывание».
7. **Проверить снимок** — загрузить файл, выбрать вид работ и технику по графику; получить технику, проверки и предупреждения. Свои снимки распознаются с подключёнными весами RF-DETR.
8. **Анализ ИИ** (если включён) — в карточке снимка из ТЗ: техника по слоям, стадия по графику и моделям, вердикты YandexGPT, «Применить исправления»; на сводке — «Анализ ИИ за день».

Работа со своим объектом через API

```
API=localhost:8000/api
# 1. проект
curl -X POST $API/projects -H "Content-Type: application/json" \
  -d '{"name": "Жилой дом, корпус 2", "object_type": "housing"}'
# 2. зоны, камеры с полигонами зон, заявленная техника (формат data/demo/*/site.json)
curl -X PUT $API/projects/3/site -H "Content-Type: application/json" -d @site.json
# 3. календарный график XLSX или CSV
curl -X POST $API/projects/3/schedule -F file=@график.xlsx
# 4. снимки камеры (время — из EXIF, имени файла или поля taken_at)
curl -X POST $API/projects/3/snapshots -F camera=CAM-01 \
  -F files=@CAM-01_2026-10-01_09-00.jpg
# 5. отклонения и отчёт
curl "$API/projects/3/deviations?day=2026-10-01"
curl "$API/projects/3/report?day=2026-10-01&format=csv" -o отчёт.csv
```

Столбцы графика определяются по заголовкам, порядок не важен: «Код» (или «Шифр», «WBS»), «Наименование работ», «Захватка» (или «Зона», «Участок»), «Начало», «Окончание», «Техника (план)» в виде «Экскаватор ×1; Самосвал ×2», «Подрядчик». Строка сопоставляется со справочником по коду (12.3.4.6 = 12.3.4-6), без кода — по названию; не нашлось — вид работ указывается вручную.

Консоль

```
cd backend
# снимок против вида работ и техники по графику
python -m app.cli check снимок.jpg -w 12.3.1 --planned "Экскаватор ×1; Самосвал ×2"
python -m app.cli rules 12.3.1 # правила методики для вида работ
python -m app.cli find котлован # поиск по справочнику
python -m app.cli ingest --project 1 --camera CAM-01 ./снимки/
python -m app.cli report --project 1 --day 2026-09-24 --csv отчёт.csv
```

Тесты

```
cd backend && python -m unittest discover -s tests # 71: движок, сценарии, API, ИИ
cd .. && python -m unittest discover -s training/tests # 6: датасет, AP50
```

Обучение детектора v2

```
pip install -r training/requirements-detector.txt
# датасеты по ссылкам из training/datasets.yaml → datasets/raw/<id>/
python training/05_build_detector_dataset.py # → datasets/oko_v2/ (COCO)
python training/06_train_detector.py --epochs 40 # → weights/rfdetr_large_v2_best.pth
python training/07_eval_conditions.py # → docs/metrics_detector_v2.md
python training/08_export_cpu.py # → weights/rfdetr_v2.onnx, замер CPU
```

Настройки

Переменная	По умолчанию	Назначение
OKO_DETECTOR	auto	auto · rf detr · onnx · http · demo
OKO_RFDETR_WEIGHTS	weights/rf detr_large_best_ema.pth	веса RF-DETR
OKO_RFDETR_CLASSES	—	порядок классов, если чекпойнт их не хранит
OKO_ONNX_MODEL	weights/rf detr_v2.onnx	модель для CPU
OKO_DEVICE	auto	auto · cpu · cuda
OKO_DETECTOR_THRESHOLD	0.3	минимальная уверенность рамки
OKO_AI_URL	http://ai:8001	адрес AI-сервиса
OKO_DATABASE_URL	SQLite в data/runtime	строка подключения PostgreSQL
OKO_DATA_DIR	data/runtime	БД SQLite и снимки
OKO_SEED_DEMO	1	загрузить демо-проекты при первом старте
OKO_MAX_UPLOAD_MB	25	лимит размера снимка
OKO_READINESS	auto	модель готовности: weights/readiness · путь · off
OKO_VLM	off	auto — Qwen3-VL под память · id модели · off
OKO_LLM_PROVIDER	off	yandex · anthropic · openai
OKO_LLM_MODEL	yandexgpt-5.1	модель LLM; для yandex — имя в AI Studio или gpt://...
OKO_YC_FOLDER_ID	—	каталог Yandex Cloud для YandexGPT
OKO_LLM_API_KEY	—	API-ключ; без него — IAM-токен сервисного аккаунта VM
OKO_AI_AUTO	0	1 — анализировать каждый новый снимок
OKO_AI_APPLY	manual	auto — применять исправления сразу

Приложение А. API

Метод и путь	Назначение
GET /api/health	Версии методики и правил, детектор и его классы, БД
GET /api/reference/equipment · profiles · work-types · rules	Методика: классы, профили, виды работ, правила
GET /api/reference/methodology.xlsx	Методика в XLSX: справочник организаторов + столбцы методики
POST /api/projects · PUT /api/projects/{id}/site	Проект; зоны, камеры с полигонами, заявленная техника
POST /api/projects/{id}/schedule · GET .../schedule	Загрузка графика (новая версия); этапы с видом работ и профилем
PATCH /api/tasks/{id}	Вручную привязать этап к виду работ или зоне

Метод и путь	Назначение
POST /api/projects/{id}/snapshots	Снимки: детекция, зоны, пересчёт; время обработки и id отклонений
GET /api/projects/{id}/snapshots · /api/snapshots/{id} · .../image	Снимки, карточка с рамками и объяснением, изображение с разметкой
GET /api/projects/{id}/summary?day=	Сводка на день: показатели, доска зон, отклонения
GET /api/projects/{id}/deviations · /api/deviations/{id}	Отклонения с зоной, этапом, расчётом и доказательствами
POST /api/deviations/{id}/review	Вердикт инженера → нарушение
GET /api/projects/{id}/violations · .../report?format=csv	Реестр нарушений, отчёт
POST /api/analyze	Снимок + вид работ → техника, проверки, предупреждения, без проекта
GET /api/ai/status	Включённые ИИ-слои: модель готовности, VLM (какая поместится в память), LLM
POST /api/snapshots/{id}/ai-review · GET ...	Запустить ИИ-анализ снимка (фоновая задача) · последний анализ
GET /api/ai/reviews/{id}?full=1	Анализ с контекстом, отправленным в LLM, и её ответом
POST /api/ai/reviews/{id}/apply · .../revert	Применить исправления с пересчётом отклонений · отменить
POST /api/projects/{id}/ai-summary?day= · GET ...	ИИ-анализ площадки за день
GET /internal/info · POST /internal/detect	Контракт AI-сервиса